Experiment 11

AIM: Develop Smart Campus Service Portal - Feature Extension (Integrate MVC + Database + API)

Code:

Database\

CampusServicePortal.sql:

```
-- Database Schema for Smart Campus Service Portal
CREATE DATABASE CampusServicesDb;
GO

USE CampusServicesDb;
GO

CREATE TABLE ServiceRequests (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    StudentName NVARCHAR(80) NOT NULL,
    EnrollmentNo NVARCHAR(20) NOT NULL,
    Department NVARCHAR(50) NOT NULL,
    ServiceCategory NVARCHAR(60) NOT NULL,
    Location NVARCHAR(100) NOT NULL,
    Description NVARCHAR(500) NOT NULL,
    Priority NVARCHAR(20) NOT NULL DEFAULT 'Medium',
    Status NVARCHAR(30) NOT NULL DEFAULT 'Pending',
    SubmittedAt DATETIME DEFAULT GETDATE()
);

-- Seed Initial Campus Service Requests
INSERT INTO ServiceRequests (StudentName, EnrollmentNo, Department, ServiceCategory, Location,
    Description, Priority, Status, SubmittedAt)
VALUES
```

```
('Anubhav Goraguddi', '92400120380', 'CSE', 'Smart Classroom A/V', 'FET Room 402', 'Projector d
    isplay flickers and HDMI input not detected during lecture.', 'High', 'In Progress', GETDAT
    E()-2),
('Sneha Sharma', '92400120388', 'IT', 'Lab Hardware', 'IT Lab 201, PC-18', 'System memory error
     on startup, machine reboots unexpectedly.', 'Medium', 'Pending', GETDATE()-1),
('Vikram Rathore', '92400120395', 'CSE', 'Network / Wi-Fi', 'Central Library 1st Floor', 'Campu
    s Wi-Fi connectivity drops frequently in study area.', 'Low', 'Resolved', GETDATE()-3);
```

Models\

ServiceRequest.cs:

```
using System.ComponentModel.DataAnnotations;

namespace SmartCampusPortal.Models;

public class ServiceRequest
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Full Name is required.")]
    [StringLength(80)]
    [Display(Name = "Student / Staff Name")]
    public string StudentName { get; set; } = string.Empty;

    [Required(ErrorMessage = "Enrollment Number is required.")]
    [StringLength(20)]
    [Display(Name = "Enrollment / ID")]
    public string EnrollmentNo { get; set; } = string.Empty;

    [Required(ErrorMessage = "Department is required.")]
    public string Department { get; set; } = string.Empty;

    [Required(ErrorMessage = "Please select a service category.")]
    [Display(Name = "Service Category")]
    public string ServiceCategory { get; set; } = string.Empty;
```

```
    [Required(ErrorMessage = "Location is required.")]
    [StringLength(100)]
    [Display(Name = "Campus Location / Room")]
    public string Location { get; set; } = string.Empty;

    [Required(ErrorMessage = "Description is required.")]
    [StringLength(500, MinimumLength = 10)]
    public string Description { get; set; } = string.Empty;

    [Required]
    public string Priority { get; set; } = "Medium";

    public string Status { get; set; } = "Pending";

    public DateTime SubmittedAt { get; set; } = DateTime.Now;
}

public class StatusUpdateDto
{
    [Required]
    public string Status { get; set; } = string.Empty;
}
```

Data\

CampusServiceDAL.cs:

```
using System.Data;
using Microsoft.Data.SqlClient;
using SmartCampusPortal.Models;

namespace SmartCampusPortal.Data;

public class CampusServiceDAL
{
    private readonly string _connectionString;
```

```
public CampusServiceDAL(string connectionString)
{
    _connectionString = connectionString;
}

public List<ServiceRequest> GetAllRequests()
{
    var list = new List<ServiceRequest>();
    using var connection = new SqlConnection(_connectionString);
    using var command = new SqlCommand("SELECT Id, StudentName, EnrollmentNo, Department, S
erviceCategory, Location, Description, Priority, Status, SubmittedAt FROM ServiceRequests O
RDER BY Id DESC", connection);

    connection.Open();
    using var reader = command.ExecuteReader();
    while (reader.Read())
    {
        list.Add(new ServiceRequest
        {
            Id = reader.GetInt32(0),
            StudentName = reader.GetString(1),
            EnrollmentNo = reader.GetString(2),
            Department = reader.GetString(3),
            ServiceCategory = reader.GetString(4),
            Location = reader.GetString(5),
            Description = reader.GetString(6),
            Priority = reader.GetString(7),
            Status = reader.GetString(8),
            SubmittedAt = reader.GetDateTime(9)
        });
    }
    return list;
}
```

```
public ServiceRequest? GetRequestById(int id)
{
    using var connection = new SqlConnection(_connectionString);
    using var command = new SqlCommand("SELECT Id, StudentName, EnrollmentNo, Department, S
erviceCategory, Location, Description, Priority, Status, SubmittedAt FROM ServiceRequests W
HERE Id = @Id", connection);
    command.Parameters.Add(new SqlParameter("@Id", SqlDbType.Int) { Value = id });

    connection.Open();
    using var reader = command.ExecuteReader();
    if (reader.Read())
    {
        return new ServiceRequest
        {
            Id = reader.GetInt32(0),
            StudentName = reader.GetString(1),
            EnrollmentNo = reader.GetString(2),
            Department = reader.GetString(3),
            ServiceCategory = reader.GetString(4),
            Location = reader.GetString(5),
            Description = reader.GetString(6),
            Priority = reader.GetString(7),
            Status = reader.GetString(8),
            SubmittedAt = reader.GetDateTime(9)
        };
    }
    return null;
}

public bool CreateRequest(ServiceRequest request)
{
    using var connection = new SqlConnection(_connectionString);
    using var command = new SqlCommand(
```

```
        "INSERT INTO ServiceRequests (StudentName, EnrollmentNo, Department, ServiceCategor
y, Location, Description, Priority, Status, SubmittedAt) VALUES (@StudentName, @EnrollmentN
o, @Department, @ServiceCategory, @Location, @Description, @Priority, @Status, @SubmittedAt
)",
        connection);

    command.Parameters.Add(new SqlParameter("@StudentName", SqlDbType.NVarChar, 80) { Value
 = request.StudentName });
    command.Parameters.Add(new SqlParameter("@EnrollmentNo", SqlDbType.NVarChar, 20) { Valu
e = request.EnrollmentNo });
    command.Parameters.Add(new SqlParameter("@Department", SqlDbType.NVarChar, 50) { Value
= request.Department });
    command.Parameters.Add(new SqlParameter("@ServiceCategory", SqlDbType.NVarChar, 60) { V
alue = request.ServiceCategory });
    command.Parameters.Add(new SqlParameter("@Location", SqlDbType.NVarChar, 100) { Value =
 request.Location });
    command.Parameters.Add(new SqlParameter("@Description", SqlDbType.NVarChar, 500) { Valu
e = request.Description });
    command.Parameters.Add(new SqlParameter("@Priority", SqlDbType.NVarChar, 20) { Value =
request.Priority });
    command.Parameters.Add(new SqlParameter("@Status", SqlDbType.NVarChar, 30) { Value = re
quest.Status });
    command.Parameters.Add(new SqlParameter("@SubmittedAt", SqlDbType.DateTime) { Value = r
equest.SubmittedAt });

    connection.Open();
    return command.ExecuteNonQuery() > 0;
}

public bool UpdateStatus(int id, string newStatus)
{
    using var connection = new SqlConnection(_connectionString);
    using var command = new SqlCommand("UPDATE ServiceRequests SET Status = @Status WHERE I
```

```
    d = @Id", connection);
        command.Parameters.Add(new SqlParameter("@Id", SqlDbType.Int) { Value = id });
        command.Parameters.Add(new SqlParameter("@Status", SqlDbType.NVarChar, 30) { Value = ne
    wStatus });

        connection.Open();
        return command.ExecuteNonQuery() > 0;
    }
}
```

Controllers\Api\

CampusServiceApiController.cs:

```
using Microsoft.AspNetCore.Mvc;
using SmartCampusPortal.Data;
using SmartCampusPortal.Models;

namespace SmartCampusPortal.Controllers.Api;

[ApiController]
[Route("api/[controller]")]
[Produces("application/json")]
public class CampusServiceApiController : ControllerBase
{
    private readonly CampusServiceDAL _dal;

    public CampusServiceApiController(IConfiguration configuration)
    {
        var connStr = configuration.GetConnectionString("DefaultConnection")
                      ?? "Server=.;Database=CampusServicesDb;Trusted_Connection=True;";
        _dal = new CampusServiceDAL(connStr);
    }

    [HttpGet]
    public ActionResult<IEnumerable<ServiceRequest>> GetRequests()
```

```
{
    return Ok(_dal.GetAllRequests());
}

[HttpGet("{id}")]
public ActionResult<ServiceRequest> GetRequest(int id)
{
    var req = _dal.GetRequestById(id);
    if (req == null) return NotFound(new { message = $"Ticket #{id} not found." });
    return Ok(req);
}

[HttpPost]
public ActionResult<ServiceRequest> SubmitRequest([FromBody] ServiceRequest request)
{
    if (!ModelState.IsValid) return BadRequest(ModelState);

    request.SubmittedAt = DateTime.Now;
    request.Status = "Pending";
    if (_dal.CreateRequest(request))
    {
        return CreatedAtAction(nameof(GetRequest), new { id = request.Id }, request);
    }
    return StatusCode(500, new { message = "Database save failed." });
}

[HttpPatch("{id}/status")]
public IActionResult UpdateStatus(int id, [FromBody] StatusUpdateDto dto)
{
    if (_dal.UpdateStatus(id, dto.Status))
    {
        return Ok(new { message = $"Status for ticket #{id} updated to {dto.Status}." });
    }
    return NotFound();
```

```
    }
}
```

Controllers\

ServicePortalController.cs:

```
using Microsoft.AspNetCore.Mvc;
using SmartCampusPortal.Data;
using SmartCampusPortal.Models;

namespace SmartCampusPortal.Controllers;

public class ServicePortalController : Controller
{
    private readonly CampusServiceDAL _dal;

    public ServicePortalController(IConfiguration configuration)
    {
        var connStr = configuration.GetConnectionString("DefaultConnection")
                      ?? "Server=.;Database=CampusServicesDb;Trusted_Connection=True;";
        _dal = new CampusServiceDAL(connStr);
    }

    public IActionResult Index()
    {
        var requests = _dal.GetAllRequests();
        return View(requests);
    }

    [HttpGet]
    public IActionResult Create()
    {
        return View(new ServiceRequest());
    }
```

```
    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Create(ServiceRequest request)
    {
        if (!ModelState.IsValid) return View(request);

        request.SubmittedAt = DateTime.Now;
        request.Status = "Pending";
        if (_dal.CreateRequest(request))
        {
            TempData["SuccessMessage"] = "Campus service ticket logged successfully.";
            return RedirectToAction(nameof(Index));
        }

        ModelState.AddModelError("", "Database error saving ticket.");
        return View(request);
    }

    public IActionResult Track()
    {
        return View();
    }
}
```

Views\ServicePortal\

Index.cshtml:

```
@model IEnumerable<SmartCampusPortal.Models.ServiceRequest>
@{
    ViewData["Title"] = "Smart Campus Service Portal";
}

<div class="row g-3 mb-4">
    <div class="col-md-3">
```

```
        <div class="card border-0 shadow-sm bg-primary text-white p-3 rounded-3">
            <div class="small text-white-50 text-uppercase fw-bold">Total Service Tickets</div>
            <h2 class="fw-bold mb-0">@Model.Count()</h2>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card border-0 shadow-sm bg-warning text-dark p-3 rounded-3">
            <div class="small text-muted text-uppercase fw-bold">Pending Action</div>
            <h2 class="fw-bold mb-0">@Model.Count(r => r.Status == "Pending")</h2>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card border-0 shadow-sm bg-info text-white p-3 rounded-3">
            <div class="small text-white-50 text-uppercase fw-bold">In Progress</div>
            <h2 class="fw-bold mb-0">@Model.Count(r => r.Status == "In Progress")</h2>
        </div>
    </div>
    <div class="col-md-3">
        <div class="card border-0 shadow-sm bg-success text-white p-3 rounded-3">
            <div class="small text-white-50 text-uppercase fw-bold">Resolved</div>
            <h2 class="fw-bold mb-0">@Model.Count(r => r.Status == "Resolved")</h2>
        </div>
    </div>
</div>

<div class="d-flex justify-content-between align-items-center mb-4">
    <div>
        <h4 class="fw-bold text-dark mb-1">Campus Maintenance &amp;amp; IT Service Requests</h4>
        <p class="text-muted small mb-0">Integrated MVC Interface + Database Operations + REST
    API Backend</p>
    </div>
    <div class="d-flex gap-2">
        <a asp-action="Track" class="btn btn-outline-primary px-3 shadow-sm">Track Ticket API</
```

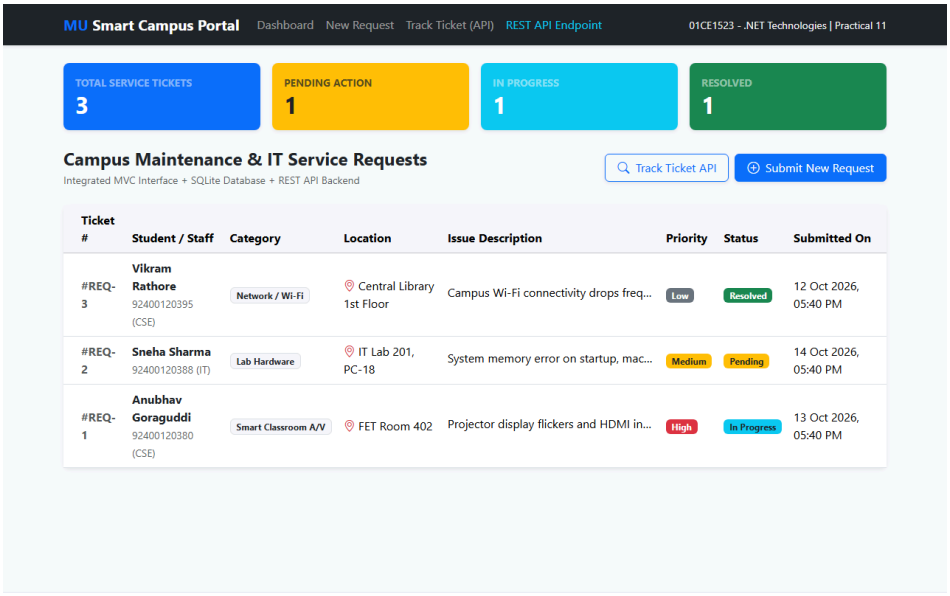
```
    a>
        <a asp-action="Create" class="btn btn-primary px-3 shadow-sm">Submit New Request</a>
    </div>
</div>

<div class="card border-0 shadow-sm rounded-3">
    <div class="card-body p-0">
        <div class="table-responsive">
            <table class="table table-hover align-middle mb-0">
                <thead class="table-light">
                    <tr>
                        <th class="ps-4">Ticket #</th>
                        <th>Student / Staff</th>
                        <th>Category</th>
                        <th>Location</th>
                        <th>Issue Description</th>
                        <th>Priority</th>
                        <th>Status</th>
                        <th>Submitted On</th>
                    </tr>
                </thead>
                <tbody>
                    @foreach (var item in Model)
                    {
                        <tr>
                            <td class="ps-4 fw-bold text-muted">#REQ-@item.Id</td>
                            <td>
                                <div class="fw-bold text-dark">@item.StudentName</div>
                                <small class="text-muted">@item.EnrollmentNo (@item.Department)
    </small>
                            </td>
                            <td><span class="badge bg-light text-dark border">@item.ServiceCate
    gory</span></td>
```

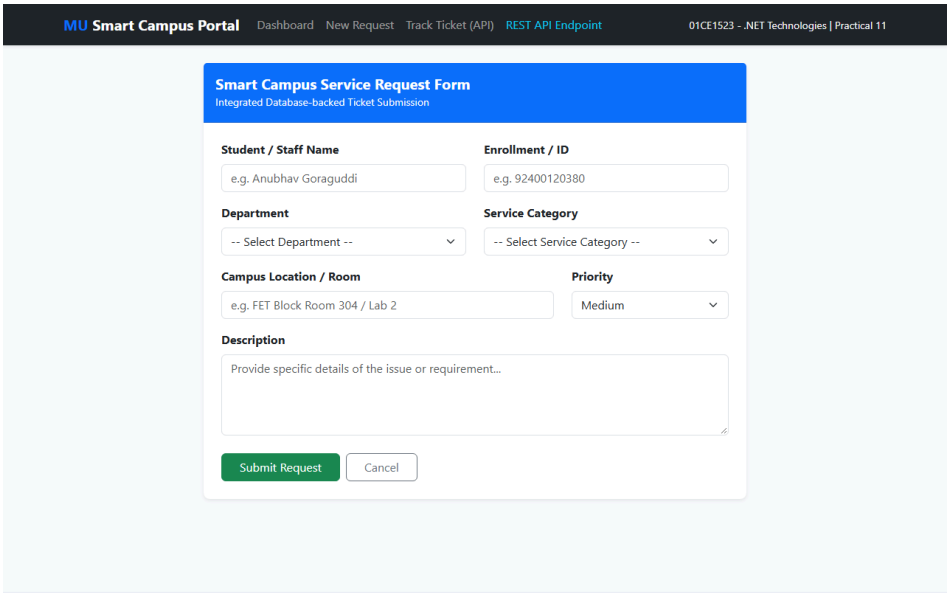
```
                        <td>@item.Location</td>
                        <td>@item.Description</td>
                        <td><span class="badge bg-danger">@item.Priority</span></td>
                        <td><span class="badge bg-info text-dark">@item.Status</span></td>
                        <td>@item.SubmittedAt.ToString("dd MMM yyyy, hh:mm tt")</td>
                    </tr>
                }
            </tbody>
        </table>
    </div>
</div>
</div>
```

Output: Screenshot:

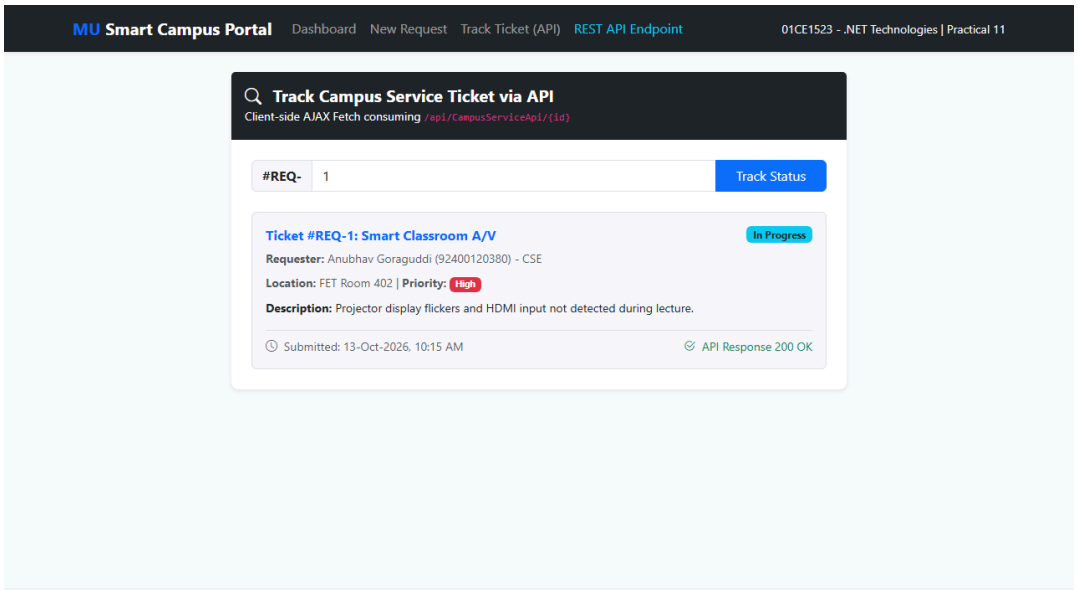
https://localhost:5001/ServicePortal (Smart Campus Service Portal Dashboard)



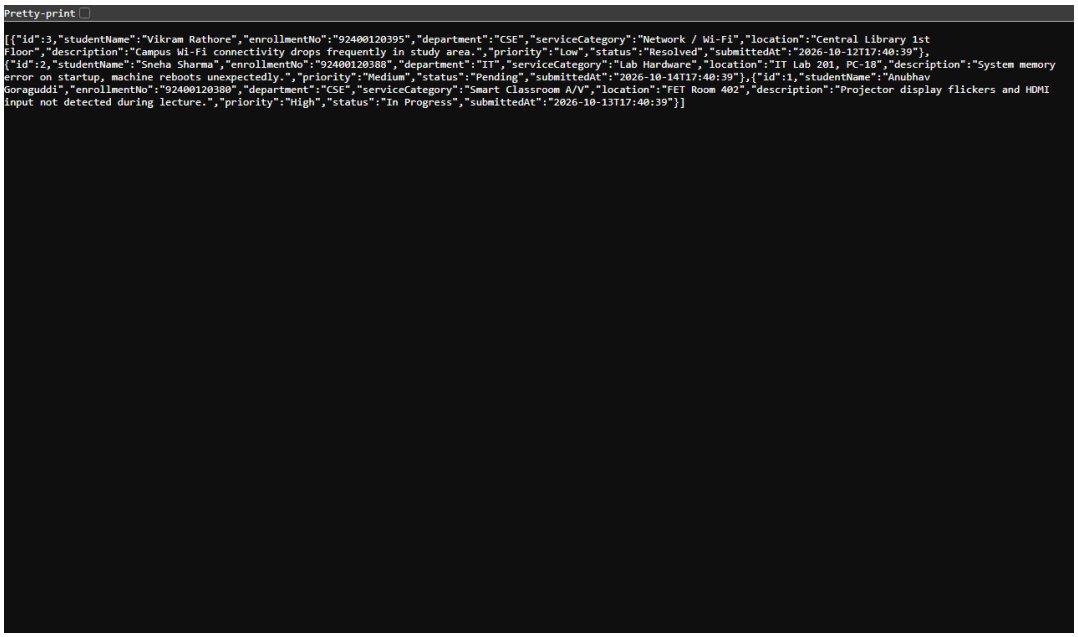
https://localhost:5001/ServicePortal/Create (Service Request Submission Form)



https://localhost:5001/ServicePortal/Track (Live Ticket Tracking using REST API AJAX)



https://localhost:5001/api/CampusServiceApi (Integrated REST API JSON Response)



GitHub URL: https://github.com/Anubhavg1123/Practical-11